

MTE 325 Lab notes

By: Yaseen Mohamed and group mates

Lab Session 1:

- After reading the schematic, Pin PA9 was the unused pin that was configured as the GPIO input for the switch. (All members discussed to achieve a conclusion, then wrote the code.)
- We were unable to read the value of the pin at first, but after debugging, we realized it was because we forgot the “volatile” keyword. (All members helped with debugging and had code running on their devices.)
- PA10 was another unused pin that was configured (as output) for the LED. (All the members discussed.)
- The LED was toggled on, wait some time, toggle off, wait some time. (All members, with different code variations on each device tested their code.)
- Motor control was not working at first, but after consulting the TAs, they helped us realize that the wrong motor number was being used in code (we were trying with M0, but once we used M1, then it worked properly). (All members attempted code; ## was able to get working after all members debugged.)

Acknowledgements: No external help, only TA assistance (TA assistance for motor control debugging)

Lab Session 2:

Exercise 5.1

1st step: (## and Yaseen built and debugged circuit, with help from all team members), (## and ## set up the oscilloscope, and Yaseen and ## connected to the circuit.)

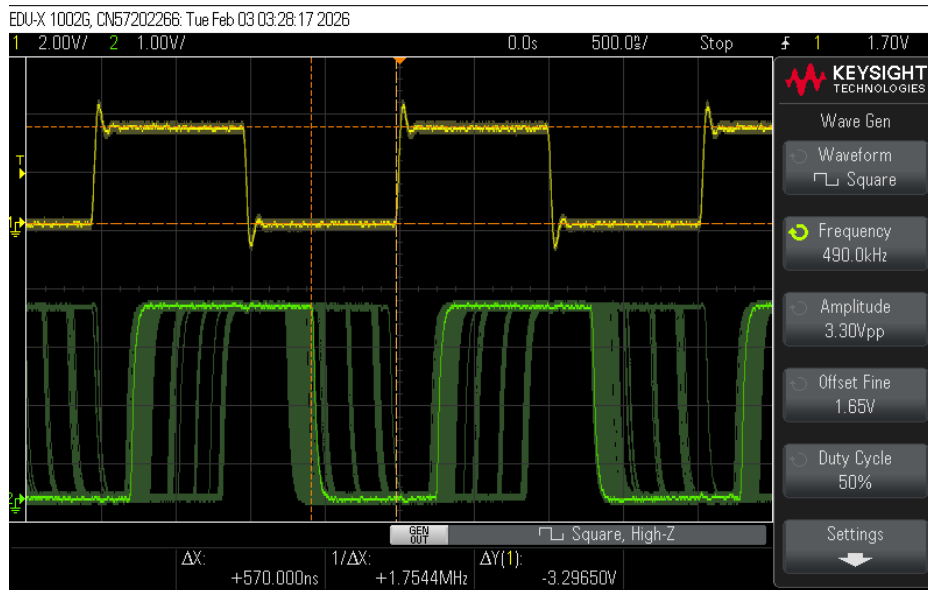
- Construct the LED circuit connected to the STM32 board using pin PA9 on the board.
- Then configured the signal generator to create a square waveform, 100hz frequency, amplitude of 3.30Vpp, and offset of 1.65V.
- Connect the positive probe of oscilloscope (signal generator) to D8 pin of the STM32 and negative probe to the ground line.

2nd step: (Tight poll code written by Yaseen and ## and debugged by rest of the team to fix errors and make it work, and ## and ## worked on oscilloscope max frequency and variation in time response analysis, with help from all members)

- We wrote code to Tight poll using a while loop, for the value at PA8
- We check if the pin is “reset”, which is low, then as soon as it is low, its “sets” or high to PA9, then repeat the same but in reverse

Problems we ran into were accidentally wiring the wrong ground and messing up the polarity of the LED connection.

In the code, the input pin was wrongfully flipped with the output when connecting to the circuit; the input and output pins were initialized to be flipped (input as output and output as input). So, the connection was flipped, and implementation was also flipped. To fix it, we read each pin separately, then reconnected the pins according to the code and compared pin initialization with the polling, reading, and resetting code.



We found the maximum frequency through increasing the frequency until the LED square wave output does not turn off before the next rising edge of the signal generator square wave.

The reliable response was determined when the LED goes low before the next rising edge of the signal generator.

Variation in response time with persistence time of 200ms: **570ns**

The reason for this variation is due to the frequency being too high for the tight polling implementation, and because of the delay added in our code, if the delay is longer, then the variation should be less.

490 khz is the maximum frequency with which we are reliably able to respond to using polling.

The code at this step:

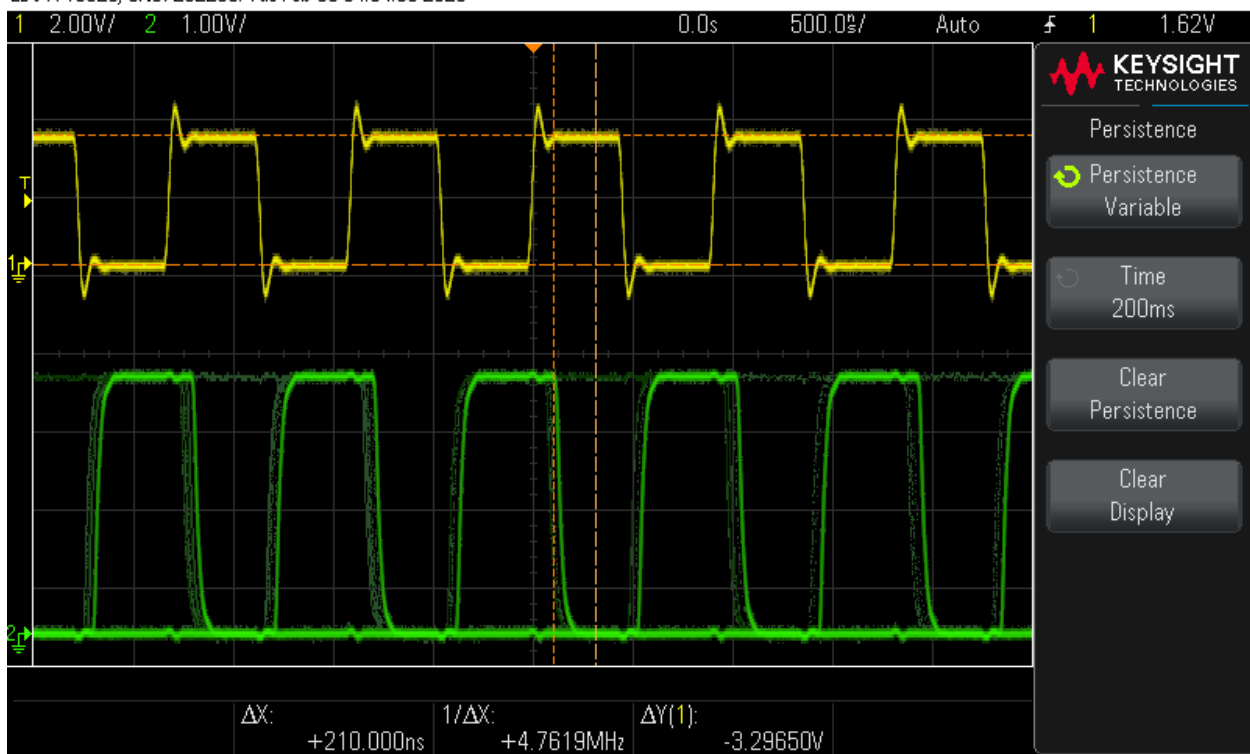
```
while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8) == GPIO_PIN_RESET) { }  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_SET); while  
(HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8) == GPIO_PIN_SET) { }  
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET);
```

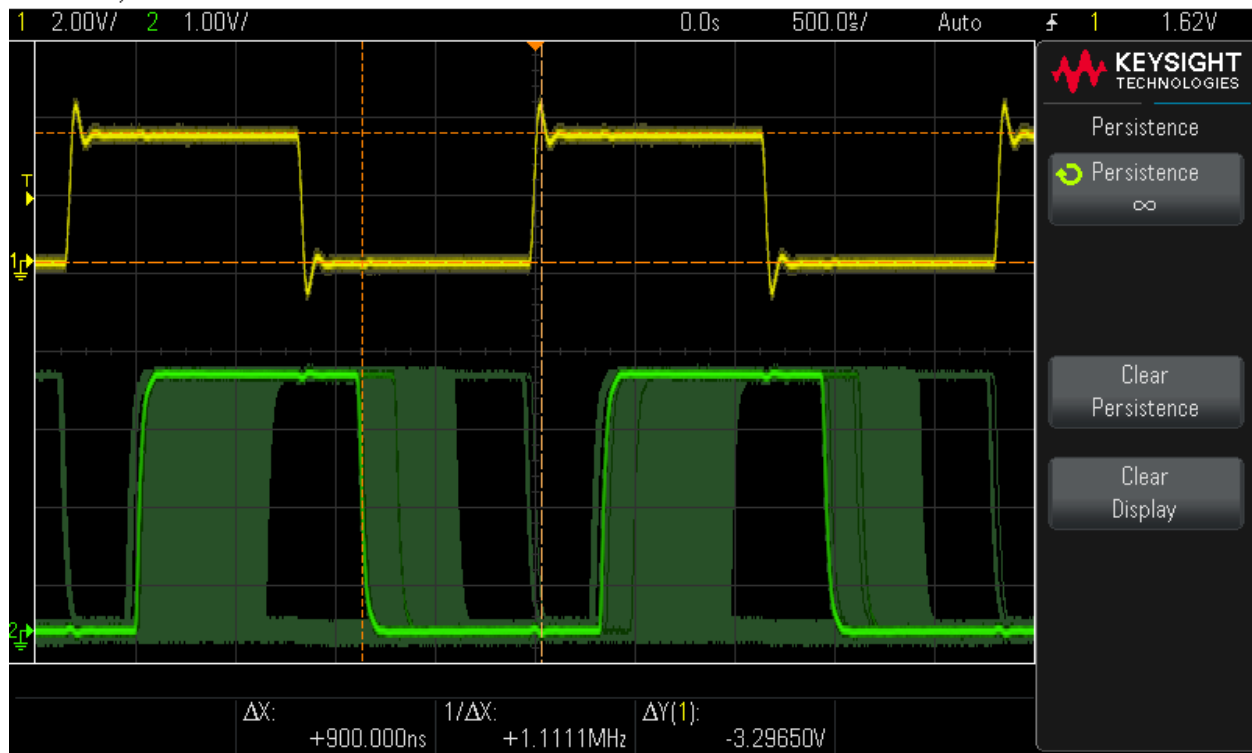
Exercise 5.2

(Interrupt code written by ## and debugged yaseen with help of the rest of the team to fix errors and make it work, and ## and ## worked on oscilloscope max frequency and variation in time response analysis, with help from all members.)

In our interrupt we need to match what we did for the polling case (rise on rise and fall on falling) (PA10 was used for the interrupt GPIO input, instead of PA8); in our interrupt it was the opposite of the polling, so when we have a rising edge interrupt, we set the LED to turn on and turn it off before the next rising edge.

EDU-X 10026, CN57202266: Tue Feb 03 04:04:35 2026





590 kHz is the maximum frequency with which we are reliably able to respond to using interrupts.

Variation in response time with persistence time of 200ms: **900ns**

Acknowledgements: No external help, only TA assistance (TA assistance with debugging errors in initial circuit connections and both codes, as well as advising that interrupts can be done on the rising edge, then tight polling on the falling edge, or interrupts on both. The TA assisted by explaining the use of the oscilloscope to get max frequency and response time variation.)

Lab Session 3:

6.1 Demonstration Based Objectives

6.1.1 Objective D1: Limit Switches

1. Objective:

The objective of this section was to implement a limit switch safety system for the two-axis motor platform. Limit switches detect when the platform reaches the end of its travel and trigger an interruption that prevents the motors from continuing to drive into the mechanical boundaries. This protects the mechanical structure and motor drivers from damage.

2. Design Approach:

Our design uses GPIO interrupts triggered on the falling edge of the limit switch signal.

The switches are configured as normally closed switches connected to 3.3 V. When the switch opens, the signal transitions from HIGH to LOW, producing a falling edge that triggers the interruption.

This approach was chosen because:

- interrupts provide immediate response
- they reduce CPU overhead compared to polling
- they ensure the motors stop quickly when the limit is reached

3. Hardware Setup

The motors were powered using an external power supply configured as:

- Supply voltage: **12 V**
- Current limit: **1.5 A**

Limit switches were connected to GPIO pins available on the motor shield connected to the STM32 board.

To determine usable pins, we:

1. Checked the existing firmware to ensure the pin was not already used.

2. Cross-referenced the motor shield schematic.
3. Verified that the pins were accessible from the shield.

From this process, the following pins were identified as available and used for limit switches and LED:

- **PA8**
- **PA9**
- **PA10**
- **PB6**
- **PB8 (for LED)**

These pins correspond to the four limit switches of the two-axis platform.

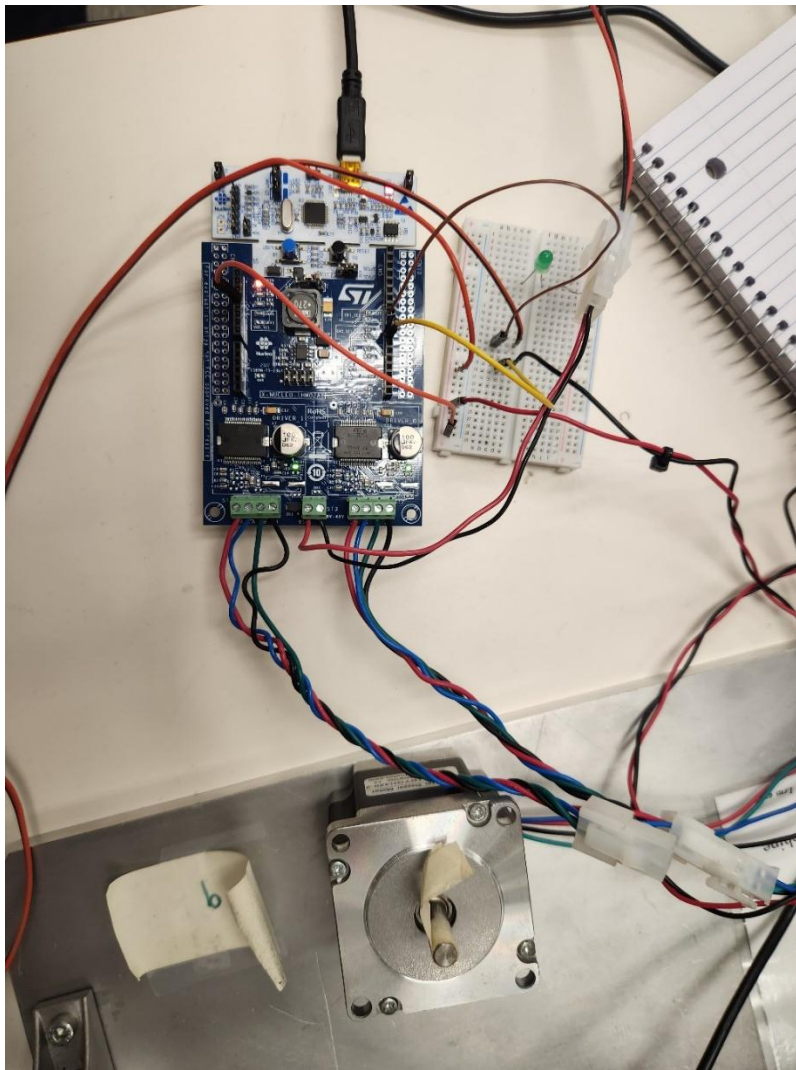


Figure: Motor Setup

4. GPIO Interrupt Configuration

Each limit switch was configured as a falling-edge triggered GPIO to interrupt.

Example configuration:

```
GPIO_InitTypeDef GPIO_InitStructure = {0};

GPIO_InitStructure.Pin = GPIO_PIN_8;
GPIO_InitStructure.Mode = GPIO_MODE_IT_FALLING;
GPIO_InitStructure.Pull = GPIO_PULLDOWN;
GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_LOW;

HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);

HAL_NVIC_SetPriority(EXTI9_5_IRQn, 2, 0);
HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);
```

This configuration allows the microcontroller to generate an interruption when the signal transitions from HIGH to LOW.

5. Interrupt Handling

When a limit switch is triggered, the interrupt service routine identifies which pin generated by the interrupt.

Example handler:

```
void EXTI9_5_IRQHandler(void)
{
    if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_9) != RESET)
    {
        HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_9);
    }

    if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_8) != RESET)
    {
        HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_8);
    }
}
```



```

}

if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_6) != RESET)
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_6);
}
}

```

The interrupt callback determines the motor response based on the activated switch.

6. Motor Control Logic

Each limit switch is associated with a specific motor direction. When a switch is activated, the motor is commanded to move away from the switch.

Example callback logic:

```

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    switch (GPIO_Pin)
    {
        case GPIO_PIN_9:
            L6470_Run(0, L6470_DIR_FWD_ID, 3000);
            break;

        case GPIO_PIN_8:
            L6470_Run(0, L6470_DIR_REV_ID, 3000);
            break;

        case GPIO_PIN_10:
            L6470_Run(1, L6470_DIR_FWD_ID, 30000);
            break;

        case GPIO_PIN_6:
            L6470_Run(1, L6470_DIR_REV_ID, 30000);
            break;
    }
}

```

This ensures that whenever a switch is triggered, the platform moves safely away from the mechanical limit.

7. Final Motor Control Strategy and Debouncing Improvement

We then worked on implementing the logic to reverse the motor direction after a switch click. The main logic follows that we know the location of each switch and the direction desired when it is pressed. Switches 1 to 4 are hardcoded at locations, and interrupt cases were set up for each switch to move the corresponding motor in the opposite direction from that switch rather than toggling.

This approach mostly eliminates the need for debouncing, since even if multiple triggers occur due to switch bouncing, the motor is always commanded to move away from the switch. Therefore, repeated triggers do not negatively affect system behavior. Additionally, when the system is at any corner, both motors will naturally move toward the opposite corner when switches are triggered, ensuring safe operation.

However, there is still a possibility that electrical noise on the lines may be interpreted as a momentary button press, which could unintentionally change motor direction. Although this is unlikely, it was considered important to address.

To mitigate this, a debounce mechanism was implemented using a custom delay function that does not interfere with interrupts:

```
void L6470_Debounce_Delay(uint32_t delay_ms){
    uint32_t t = delay_ms*20000;
    for (int i = 0; i < t; i++) {}
};
```

This function approximates a delay using a busy loop, avoiding the use of HAL_Delay, which previously caused interrupt conflicts.

The debounce was applied within the interrupt callback as follows:

```
case GPIO_PIN_10: // vertical axis top switch
    L6470_Debounce_Delay(debounce_delay_ms);
    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_10) == GPIO_PIN_RESET && fault_detected == false)
    {
        L6470_Run(1, L6470_DIR_FWD_ID, 30000);
    }
    break;
```

This implementation ensures that button presses are only registered when they are stable, preventing false triggering due to noise.

Finally, an additional safety improvement was identified: when an interrupt is triggered, the system should check if both switches on the same axis are pressed. If so, this indicates a fault condition, and both motors should be stopped immediately to prevent damage to the system.

8. Edge Case Handling

Several edge cases were considered and addressed to ensure safe and reliable operation of the system. Since the system interacts with physical hardware, unexpected conditions such as switch noise, incorrect wiring, or startup states can lead to unsafe behavior if not properly handled.

One important consideration is that limit switches may not produce perfectly clean signals. Due to mechanical bouncing or electrical noise, a switch may briefly toggle multiple times when pressed. To mitigate this, a debounce mechanism was implemented using a custom delay function that does not interfere with interrupts. This ensures that only stable switch activations are processed by the system.

Another critical edge case occurs when both limit switches on the same axis are triggered simultaneously. Under normal operation, this condition should not occur, as the platform cannot physically be at both ends of an axis at the same time. Therefore, this scenario is treated as a fault condition, which may indicate:

- incorrect wiring
- hardware malfunction
- mechanical obstruction or misalignment

When this condition is detected, both motors are immediately stopped using a hard stop command, and a fault flag is set. This prevents further motion and protects the system from potential damage.

Additionally, the system must handle cases where a limit switch is already triggered at startup. This can happen if the platform is positioned at the end of its travel when power is applied. In this situation, the system automatically commands the motor to move in the opposite direction, ensuring that the platform returns to a safe operating region.

Another potential issue is false triggering due to electrical noise on the input lines. Although unlikely, this could cause unintended motor direction changes. To address this,

the debounce logic ensures that the input signal is stable before executing any motor command.

Overall, these edge case considerations improve the robustness and safety of the system by ensuring that abnormal conditions are detected and handled appropriately.

9. Both switches triggered on one axis

If both limit switches on the same axis are detected simultaneously, this indicates a possible system fault such as:

- wiring error
- mechanical obstruction
- sensor malfunction

In this situation, both motors are stopped and a fault flag is set:

```
L6470_HardStop(0);  
L6470_HardStop(1);  
fault_detected = true;
```

A UART message is also transmitted to notify the user of the fault.

10. Limit switch triggered at startup

If the system starts while touching a limit switch, the motor is automatically commanded to move away from the activated switch to return the platform to a safe operating region.

11. Testing

The system was tested using unmounted motors and manual activation of the limit switches.

Testing verified that:

- interrupts were triggered correctly
- the motor responded immediately to limit switch activation
- the motor moved away from the activated switch
- fault detection worked when both switches were triggered

The tests confirmed reliable operation of the limit switch safety system.

In addition to testing with unmounted motors, the machine was also tested on both axes to verify that motion could be safely stopped and correctly redirected when the limit switches were triggered. This confirmed that the limit switch logic worked correctly during actual machine operation and that both motors could operate safely within the system boundaries.

Lab Session 4:

6.1.2 Objective D2: ADC Characterization

1. Objective:

The objective of this section was to characterize the Analog-to-Digital Converter (ADC) on the STM32 board and use it to read the analog voltage from a potentiometer. The ADC readings were then used to control the speed of one of the motors.

2. Potentiometer Connection & ADC Pin Selection

We figured out how to connect the potentiometer to the board as shown in the figure below.



Figure: Potentiometer Pinout

A potentiometer was connected as a **voltage divider** to generate an adjustable analog voltage.

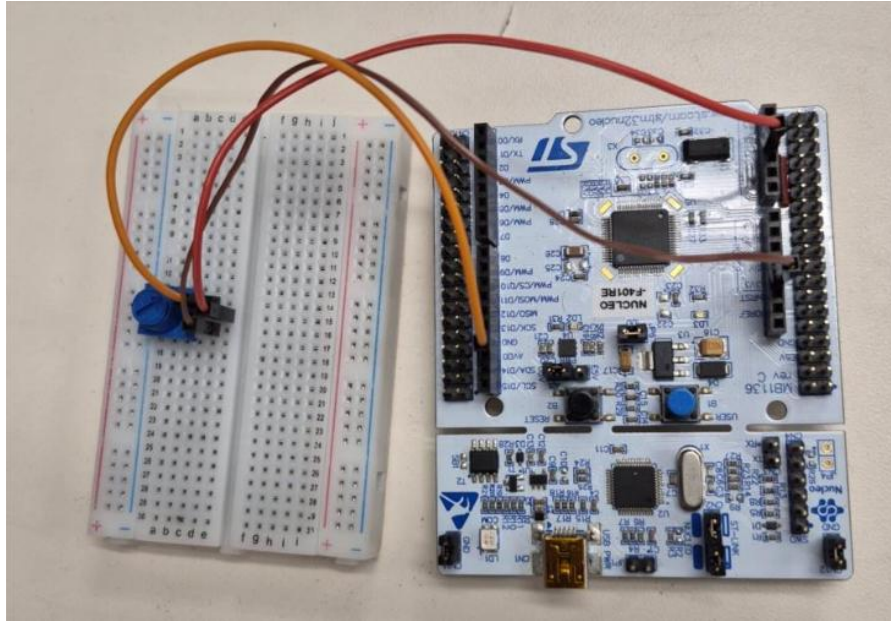


Figure: Potentiometer setup

After examining the schematic, **PB0** was identified as an unused analog-capable pin and was selected as the ADC input.

28	PA0	A0	1
30	PA1	A1	2
32	PA4	A2	3
34	PB0	A3	4
36	PC1	A4	5
38	PC0	A5	6

Figure: STM32 schematic showing ADC input connection (PB0)

3. ADC Configuration and Activation

The internal ADC was configured by modifying the `MX_ADC1_Init()` function.

Key configuration parameters:

```

hadc1.Instance = ADC1;
hadc1.Init.ClockPrescaler = ADC_CLOCKPRESCALER_PCLK_DIV4;
hadc1.Init.Resolution = ADC_RESOLUTION_8B;
hadc1.Init.ScanConvMode = DISABLE;
hadc1.Init.ContinuousConvMode = ENABLE;
hadc1.Init.DiscontinuousConvMode = DISABLE;
hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;

```

```
hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;  
hadc1.Init.NbrOfConversion = 1;  
hadc1.Init.DMAContinuousRequests = DISABLE;  
hadc1.Init.EOCSelection = ADC_EOC_SEQ_CONV;
```

The selected ADC channel was ADC_CHANNEL_8, corresponding to pin PB0, which was previously identified as the analog input for the potentiometer.

Activation of ADC

The ADC was started using HAL_ADC_Start(), enabling continuous conversion of the input signal.

4. ADC Resolution Selection Design Decisions:

What resolution did you choose? Why?

The Nucleo board supports 6-, 8-, 10-, and 12-bit ADC resolution. Each additional bit increases the number of discrete levels according to 2^N , allowing finer representation of the analog input voltage.

In this application, a potentiometer is used to manually control motor speed. Since the potentiometer has a small knob and is adjusted by hand, the achievable precision is limited by human control rather than hardware capability.

A 12-bit resolution provides 4096 levels and 10-bit provides 1024 levels, both of which exceed the practical resolution achievable through manual adjustment. A 6-bit resolution provides only 64 levels, which is too coarse and would result in large step changes in motor speed.

An 8-bit resolution provides 256 levels, which offers a good balance between control precision and smoothness. This allows for gradual and stable changes in motor speed while remaining within the limits of human input.

Additionally, due to the small travel distance of the motor, coarse resolution could result in abrupt speed transitions, potentially causing unstable or erratic motion. The 8-bit resolution provides sufficiently smooth transitions and was therefore selected as the optimal configuration.

Therefore, an 8-bit resolution was selected for this application. An 8-bit resolution provides 256 discrete steps, offering a good balance between precision and smooth motor control.

5. Conversion Mode

What conversion mode? Why?

The ADC was configured in continuous conversion mode so that the analog signal could be sampled continuously. This allows the motor speed to update in real time as the potentiometer is rotated.

6. Sampling Time Selection

What is your sampling time? Why?

We configured the ADC1 sampling time to 15 cycles. At 21 MHz, one cycle is 47.6ns. If a sampling time of 3 cycles is used, then the total sampling time is 142.8ns. The following calculations were done to find the required time for an 8-bit accuracy, while assuming the voltage must settle within 0.5 LSB:

$$\begin{aligned}V_c(t) &= V_{in} \left(1 - e^{-\frac{t}{\tau}}\right) \\V_{error} &= V_{in} - V_c(t) = V_{in} (e^{-\frac{t}{\tau}}) \\ \frac{V_{error}}{V_{in}} &= e^{-\frac{t}{\tau}} = \frac{0.5}{2^n} \\ t &= 6.24 \tau\end{aligned}$$

This means that the RC circuit needs about 6.2 time constants to settle within 0.5 LSB for 8-bit accuracy.

Now solving for the time constant, the max is used, based on the full resistance value from the potentiometer (10kohm) being in series with the internal resistance of (6kohm), totalling 16kohm:

$$\tau = RC = 16000ohm * (4 * 10^{-12}Farad) = 64ns$$

For 8-bit accuracy, we require approximately 6.2 time constants to allow the internal capacitor to charge within 0.5 LSB of the input voltage.

$$64ns * 6.2 = 384ns$$

Now, since the sampling time of 3 cycles (142.8ns) is less than the calculated required time for the voltage to settle, this means that the sampling time must occur over a greater number of cycles. The next sampling time cycle constant available in the HAL is ADC_SAMPLETIME_15CYCLES. With 15 cycles instead, the sampling time becomes 714ns, which is greater than the required time of 384ns. By choosing 15 cycles (714ns), we

ensured the internal capacitor has more than enough time to fully charge, which prevents 'ghosting' or lagged readings when we adjust the motor speed in real-time.

7. ADC Clock configuration

What happens when you change your clock settings?

The ADC clock prescaler was configured to divide the APB2 clock by four. According to the STM32F401xE datasheet, the maximum allowable ADC clock frequency is 32 MHz. The APB2 clock operates at 84 MHz; therefore, using a prescaler of 2 would result in an ADC clock of 42 MHz, which exceeds the specified limit. By selecting a prescaler of 4, the ADC clock is reduced to:

$$84 \text{ MHz} / 4 = 21 \text{ MHz}$$

This value lies safely within the allowable operating range while still providing sufficiently fast conversions.

Additionally, since the potentiometer input changes relatively slowly and the system uses only 8-bit resolution, extremely high ADC sampling speeds are not required. A prescaler of 4 provides an effective balance between conversion speed and measurement stability, ensuring reliable and accurate readings without unnecessary performance overhead.

8. Verifying ADC Operation

How can you ensure your ADC is enabled and working?

Now inside the main loop, we add the condition to check if the ADC conversion is complete, then read its value (0-255). The read return value is printed to ensure the range of values is properly read and convert to motor speed value later. The code below was used in main to read and print the ADC value:

The ADC was started using:

```
HAL_ADC_Start(&hadc1);
```

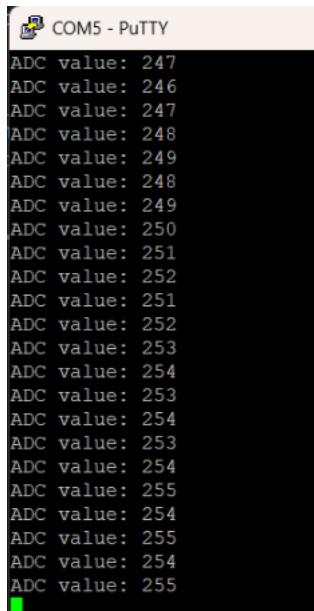
The program continuously checked the ADC conversion status:

```
HAL_ADC_PollForConversion(&hadc1, 10);
```

When a conversion completed, the ADC value was read:

```
ADC_raw = (uint16_t)HAL_ADC_GetValue(&hadc1);
```

The value was transmitted through UART to verify the ADC readings. After verifying the ADC value is read properly, and printed as shown below



```
COM5 - PuTTY
ADC value: 247
ADC value: 246
ADC value: 247
ADC value: 248
ADC value: 249
ADC value: 248
ADC value: 249
ADC value: 250
ADC value: 251
ADC value: 252
ADC value: 251
ADC value: 252
ADC value: 253
ADC value: 254
ADC value: 253
ADC value: 254
ADC value: 253
ADC value: 254
ADC value: 255
ADC value: 254
ADC value: 255
ADC value: 254
ADC value: 255
```

Figure: Output Data from COM5 Port

The prints were removed, and the ADC value was converted to a speed value range for the vertical motor (10000 to 50000), and the motor speed was set as a global variable, and the motor speed would immediately change to the new speed as the ADC value changes.

As the potentiometer resistance is changed that results in the equivalent change in the ADC output which has been mapped linearly to an output between 0 and 255.

9. ADC Linearity Test

Ideally your ADC is linear. How accurate is it?

To characterize the ADC, measurements were taken at 11 different potentiometer voltages. The results showed a linear relationship with $R^2 \approx 1$, confirming that the ADC behaves linearly within the tested range.

Testing the ADC linearity to check if it's working. To characterize the ADC, measurements were taken at 11 different potentiometer voltages. For calibration of ADC, we have V_Potentiometer vs ADC out graph. Taking 11 data points from Voltage of analog.

Table: data table of Analog voltage vs ADC count output

ANALOG voltage	DIGITAL output
0	0
0.3	22
0.6	45
0.9	69
1.2	92
1.5	115
1.8	138
2.1	161
2.4	185
2.7	207
3	231
3.3	255

DIGITAL output vs. ANALOG voltage

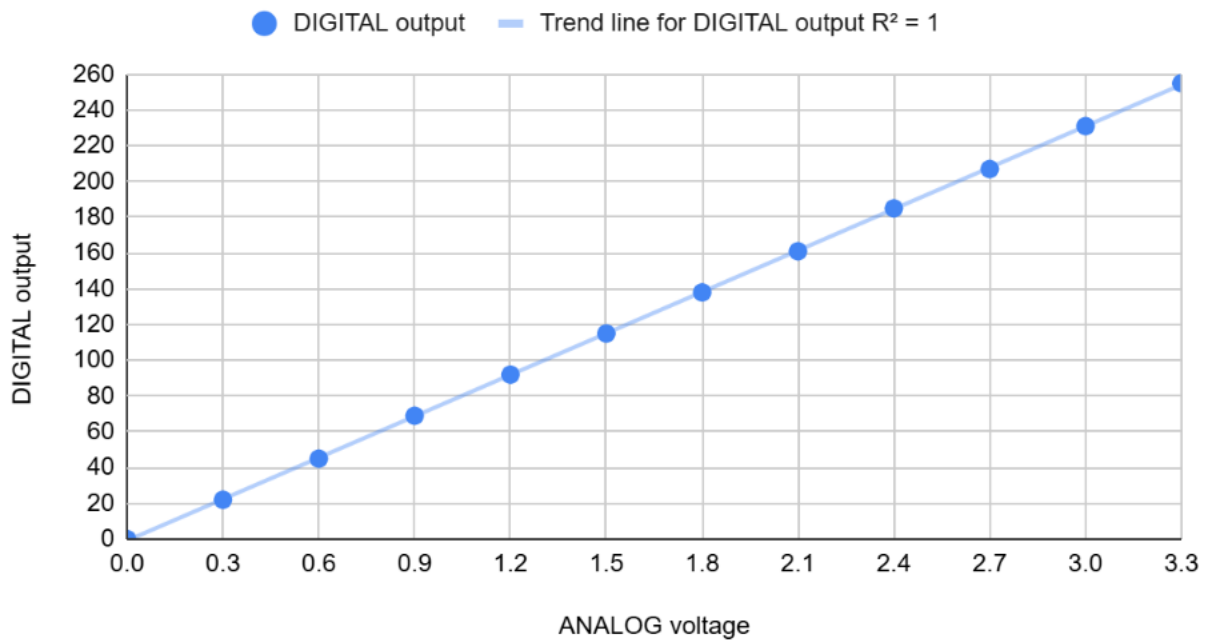


Figure: Graph of DIGITAL_output vs ANALOG_voltage

The results showed a linear relationship with $R^2 \approx 1$, confirming that the ADC behaves linearly within the tested range.

The results showed a linear relationship with $R^2 \approx 1$, confirming that the ADC behaves linearly within the tested range. It is very accurate. The ADC was configured with 8-bit

resolution and a reference voltage of 3.3 V. This results in 256 discrete quantization levels. The voltage resolution of the ADC is therefore

$$3.3V/256 \approx 12.9mV$$

per step.

These ADC characterization results were reasonable because the measured digital output increased linearly with input voltage across the tested range. One limitation of the experiment was that small fluctuations in ADC readings were observed during real-time reading, especially when the potentiometer was held between intermediate positions. If this experiment were repeated, more data points could be collected and the sampling process could be automated to improve repeatability and better quantify small ADC variations.

10. ADC Resolution and Error

What is the error in your ADC?

With an 8-bit resolution and reference voltage of **3.3 V**, the voltage resolution is:

$$3.3\text{ V} / 256 \approx 12.9\text{ mV per step}$$

The theoretical quantization error of the ADC is:

$$\pm 0.5\text{ LSB} \approx \pm 6.45\text{ mV}$$

This represents the inherent measurement uncertainty of the ADC due to quantization. The experimental results showed a linear relationship between the potentiometer voltage and the digital ADC output, indicating that the ADC behaved as expected within the limits of quantization error. Since the measured response was linear, no significant integral nonlinearity (INL) was observed within the experimental resolution.

For potentiometer characterization, it is a voltage divider circuit, graph v pot_resistance. This graph shows the expected nonlinear relationship between output voltage and resistance division, while the measured ADC voltage-output graph confirmed that the ADC response itself was linear over the tested operating range.

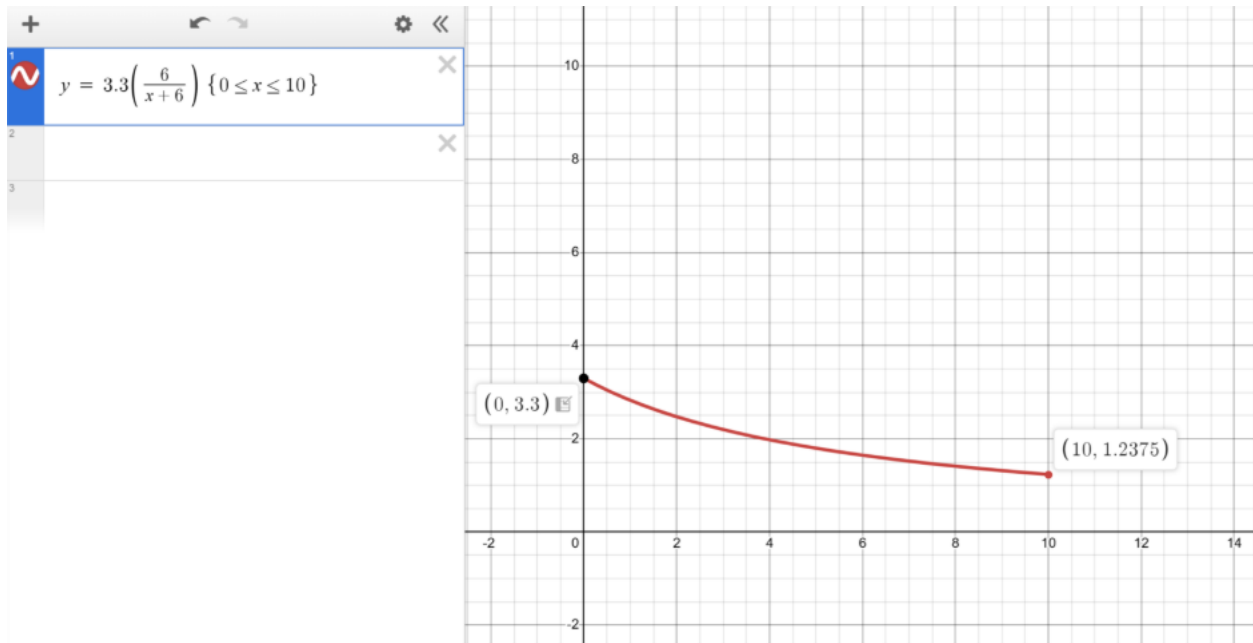


Figure: Graph V pot_resistance

11. Mapping ADC Value to Motor Speed

Define and carry out a calibration procedure? Does it behave linearly? How can you map the ADC readings to motor speed?

The ADC reading ranges from 0 to 255 for an 8-bit configuration. For the initial single-motor ADC test, the ADC reading was linearly mapped to a motor speed range between 10000 and 50000. The mapping equation used is:

$$\text{motor_speed} = 10000 + \frac{\text{ADC}_{\text{raw}}(50000 - 10000)}{255}$$

This mapping ensures:

- minimum motor speed when $\text{ADC}_{\text{raw}} = 0$
- maximum motor speed when $\text{ADC}_{\text{raw}} = 255$
- smooth and continuous speed variation for intermediate values

Calibration and Linearity

To calibrate the ADC, a relationship between the input analog voltage and the digital ADC output was established using measured data points. A graph of input voltage versus ADC output was generated, which showed a linear relationship.

Since the potentiometer operates as a voltage divider, its resistance changes linearly with rotation, producing a proportional change in output voltage. This results in a corresponding linear change in the ADC reading.

Because of this linear behavior, a simple linear mapping function can be used to convert ADC readings directly into motor speed values. For values between measured data points, interpolation can be applied if higher precision is required.

This confirms that the ADC behaves linearly within the operating range and is suitable for real-time motor control.

Real-Time Motor Speed Control

After verifying ADC functionality, a global variable was introduced to store the motor speed. The main control loop was modified to continuously read the ADC value and update the motor speed accordingly.

The system operates as follows:

1. The ADC value is continuously polled
2. The new ADC value is compared with the previous value
3. If the value has changed, a new motor speed is calculated using the mapping equation
4. The motor speed is updated only if:
 - no fault condition is detected
 - the motor is currently running
 - the new ADC value is an even number (ensures the potentiometer fluctuation is ignored by using only half the steps, to minimize speed adjustments)

The motor direction remains unchanged during speed updates and is later maintained using the synchronized motor state variables described later in the motor characterization section.

Implementation (was later modified to fix issues and work better with interrupts, and use both ADCs with scan conversion mode, this was just a basic implementation just for speed control)

```
while (1)
{
    USART_CheckAppCmd();

    testing = HAL_ADC_PollForConversion(&hadc1, 10);
    if (testing == HAL_OK)
    {
        ADC_raw = (uint16_t)HAL_ADC_GetValue(&hadc1);
        if (ADC_raw != ADC_old)
        {
            ADC_old = ADC_raw;

            motor_1_speed = 10000 + ADC_raw * (50000 - 10000) / 255;

            if (fault_detected == false &&
                L6470_CheckStatusRegisterFlag(1, BUSY_ID) == 0)
            {
                L6470_Run(1,
                    L6470_CheckStatusRegisterFlag(1, DIR_ID),
                    motor_1_speed);
            }
        }
    }
}
```

System Behavior

The motor speed updates in real time as the potentiometer is rotated. This provides:

- smooth and continuous control of motor speed
- immediate system response to user input
- stable operation without unnecessary updates

12. Additional Potentiometer for other motor

To add a second potentiometer, we needed another ADC on another pin, and since, according to the data sheet, the MCU only has ADC1, we had to use the same ADC1, and a different pin (different channel). Therefore pin A0, which is on channel 0 (in addition to our first pin B0 on channel 8). Then the initialization needed to be slightly modified to read both channels. ScanConvMode was enabled instead of the ContinuousConvMode, and NbrOfConversion was modified to 2. The new pin was configured, then the channels were configured as follows:

```
sConfig.Channel = ADC_CHANNEL_0; // PA0  
  
sConfig.Rank = 1;  
  
sConfig.SamplingTime = ADC_SAMPLETIME_15CYCLES;  
HAL_ADC_ConfigChannel(&hadc1, &sConfig);  
  
  
sConfig.Channel = ADC_CHANNEL_8; // PB0  
  
sConfig.Rank = 2;  
  
sConfig.SamplingTime = ADC_SAMPLETIME_15CYCLES;  
HAL_ADC_ConfigChannel(&hadc1, &sConfig);
```

The next step was to properly read both channels and adjust the motor speeds. The same polling code was used, but modified to read the first pin, then the second pin, separately. It requires the HAL_ADC_Start function to be called to start each read, as opposed to the continuous conversion mode, and then each pin has to be asked to convert a value using HAL_ADC_PollForConversion, then it can be read from it. To

ensure both ADCs can control both motors simultaneously, one function that is being polled reads the first channel, and adjusts the speed as needed, then reads the second channel and adjusts the other motor speed as needed.

Implementation (comments removed for simplicity) (check the limit_switch_yaseen branch for the original code)

```
void ADC1_to_Motor(ADC_HandleTypeDef* hadc)
{
    static volatile uint32_t ADC_raw_0 = 0;
    static volatile uint32_t ADC_raw_1 = 0;
    static volatile uint32_t ADC_old_0 = 5000;
    static volatile uint32_t ADC_old_1 = 5000;
    static uint8_t init_done = 0;
    if (!init_done)
    {
        volatile uint32_t adc_bits = 0;
        switch (hadc->Init.Resolution)
        {
            case ADC_RESOLUTION_12B: adc_bits = 12; break;
            case ADC_RESOLUTION_10B: adc_bits = 10; break;
            case ADC_RESOLUTION_8B: adc_bits = 8; break;
            case ADC_RESOLUTION_6B: adc_bits = 6; break;
            default: adc_bits = 0; break;
        }
        ADC_old_0 = (uint16_t)(1U << adc_bits);
        ADC_old_1 = (uint16_t)(1U << adc_bits);
        init_done = 1;
    }
    if (HAL_ADC_Start(hadc) != HAL_OK)
    {
        return;
    }
    static volatile uint32_t fail_count_0 = 0;
    static volatile uint32_t fail_count_1 = 0;
    HAL_StatusTypeDef adc_status = HAL_ADC_PollForConversion(hadc, 10);
    if (adc_status == HAL_OK)
    {
        ADC_raw_0 = (uint16_t)HAL_ADC_GetValue(hadc);
        fail_count_0 = 0;
    }
    else
    {

```

```

    fail_count_0++;
    if (ADC_old_0 == 5000 || fail_count_0 > 50)
    {
        ADC_raw_0 = 0;
        fail_count_0 = 0;
    }
}
if (HAL_ADC_PollForConversion(hadc, 10) == HAL_OK)
{
    ADC_raw_1 = (uint16_t)HAL_ADC_GetValue(hadc);
    fail_count_1 = 0;
}
else
{
    fail_count_1++;
    if (ADC_old_1 == 5000 || fail_count_1 > 50)
    {
        ADC_raw_1 = 0;
        fail_count_1 = 0;
    }
}
if (ADC_raw_0 != ADC_old_0 && ADC_raw_0 % 2 == 0)
{
    ADC_old_0 = ADC_raw_0;
    motor_0_speed = motor_0_speed_min +
        ADC_raw_0 * (motor_0_speed_max - motor_0_speed_min) / 255;
    if (fault_detected == false)
    {
        if (manual_control == true && manual_stopped[MOTOR_0] == false)
        {
            No_Interruption_Delay_ms(10);
            if (manual_control == true && manual_stopped[MOTOR_0] == false)
            {
                L6470_Run(MOTOR_0, motor_direction[MOTOR_0], motor_0_speed);
                No_Interruption_Delay_ms(10);
                if (motor_direction[MOTOR_0] !=
L6470_CheckStatusRegisterFlag(MOTOR_0, DIR_ID))
                {
                    L6470_Run(MOTOR_0, motor_direction[MOTOR_0], motor_0_speed);
                    No_Interruption_Delay_ms(20);
                }
            }
            if (manual_stopped[MOTOR_0] == true)
            {

```

```

        L6470_HardStop(MOTOR_0);
    }
}
else if (manual_control == false)
{
    L6470_Run(MOTOR_0, motor_direction[MOTOR_0], motor_0_speed);
    No_Interrupt_Delay_ms(10);
    if (motor_direction[MOTOR_0] != L6470_CheckStatusRegisterFlag(MOTOR_0,
DIR_ID))
    {
        L6470_Run(MOTOR_0, motor_direction[MOTOR_0], motor_0_speed);
        No_Interrupt_Delay_ms(20);
    }
}
}
}
if (ADC_raw_1 != ADC_old_1 && ADC_raw_1 % 2 == 0)
{
    ADC_old_1 = ADC_raw_1;
    motor_1_speed = motor_1_speed_min +
        ADC_raw_1 * (motor_1_speed_max - motor_1_speed_min) / 255;
    if (fault_detected == false)
    {
        if (manual_control == true && manual_stopped[MOTOR_1] == false)
        {
            No_Interrupt_Delay_ms(10);
            if (manual_control == true && manual_stopped[MOTOR_1] == false)
            {
                L6470_Run(MOTOR_1, motor_direction[MOTOR_1], motor_1_speed);
                No_Interrupt_Delay_ms(10);
                if (L6470_CheckStatusRegisterFlag(MOTOR_1, DIR_ID) !=
motor_direction[MOTOR_1])
                {
                    L6470_Run(MOTOR_1, motor_direction[MOTOR_1], motor_1_speed);
                    No_Interrupt_Delay_ms(20);
                }
                if (manual_stopped[MOTOR_1] == true)
                {
                    L6470_HardStop(MOTOR_1);
                }
            }
        }
    }
    else if (manual_control == false)
    {

```

```

        L6470_Run(MOTOR_1, motor_direction[MOTOR_1], motor_1_speed);
        No_Interrupt_Delay_ms(10);
        if (L6470_CheckStatusRegisterFlag(MOTOR_1, DIR_ID) !=
motor_direction[MOTOR_1])
        {
            L6470_Run(MOTOR_1, motor_direction[MOTOR_1], motor_1_speed);
            No_Interrupt_Delay_ms(20);
        }
    }
}
}
}
}
}

```

13. ADC Edge cases

- The ADC reading often fluctuated slightly, so filtering was needed to keep the motor speed from constantly changing, by making it only change for even ADC values.
- ADC speed control could conflict with limit-switch interrupts, so the motor direction and motor state were rechecked before finalizing each speed update to ensure they had not been changed by an interrupt.
- At startup, the ADC input could already be at any random value, so the initial reading had to be handled separately before the motor initialization.

14. Summary

The ADC characterization demonstrated that the STM32 ADC provides accurate and linear conversion of the analog potentiometer signal. The ADC readings were successfully used to dynamically control motor speed for both motors simultaneously. The selected configuration ensured adequate sampling time, stable operation within the ADC frequency limits, and sufficient resolution for smooth motor control, which was later slightly modified to read 2 channels.

6.1.3 Objective D3: Motor Characterization

1. Objective

The objective of this section was to characterize the behavior of the motors and determine appropriate control strategies for speed, direction, and safe operation. This includes identifying suitable speed ranges, mapping ADC input to motor control, and ensuring reliable operation under different control modes.

2. Motor Control and System Operation

We have both our ADCs and limit switches working. The next step is to identify how the motors can be controlled and in what patterns.

This section evaluates how ADC-based speed control and limit switch feedback are integrated to achieve reliable and safe motor operation

3. Available Motor Control Parameters

The control commands available to us are:

```
#define L6470_MAX_POSITION      (0x1FFFFF)    //!< Max position
#define L6470_MIN_POSITION      (-0x200000)    //!< Min position
#define L6470_POSITION_RANGE    ((uint32_t)(L6470_MAX_POSITION -
L6470_MIN_POSITION))    //!< Position range
#define L6470_MAX_SPEED         (0xFFFFF)     //!< max value of SPEED
#define L6470_MAX_ACC           (0xFFF)       //!< max value of ACC
#define L6470_MAX_DEC           (0xFFF)       //!< max value of DEC
#define L6470_MAX_MAX_SPEED     (0x3FF)       //!< max value of MAX_SPEED
#define L6470_MAX_MIN_SPEED     (0xFFF)       //!< max value of MIN_SPEED
#define L6470_MAX_FS_SPD        (0x3FF)       //!< max value of FS_SPD
#define L6470_MAX_INT_SPEED     (0x3FFF)      //!< max value of INT_SPEED
#define L6470_MAX_ST_SLP        (0xFF)        //!< max value of ST_SLP
#define L6470_MAX_FN_SLP_ACC     (0xFF)       //!< max value of FN_SLP_ACC
#define L6470_MAX_FN_SLP_DEC     (0xFF)       //!< max value of FN_SLP_DEC
#define L6470_MAX_OCD_TH        (0xF)        //!< max value of OCD_TH
#define L6470_MAX_STALL_TH       (0x7F)       //!< max value of STALL_TH
```

The initial way to control the motor was using the built in terminal, the built in commands for the motor for forward, backward and input speed. Our plan is now to map this to input digital value from the analog voltage of a potentiometer that goes through an ADC.

4. Mapping of Potentiometer to Motor Control

The potentiometer position to ADC conversion follows the same linear mapping as done before (based on the graph of 11 steps of voltage and corresponding digital output from the ADC). The ADC output ranges from 0 to 255.

We mapped:

- 0–255 → 0 - 7000 for the horizontal axis
- 0–255 → 0 - 70000 for the vertical axis

This mapping can be expressed as:

$$motor_{speed} = \frac{ADC_{raw}}{255} \times max_speed$$

The difference in speed ranges is due to the mechanical structure:

- The horizontal motor slides across a short rail with wheels
- The vertical motor uses a lead screw, which requires significantly higher rotation to achieve the same displacement

The values are mapped using the ADC characterization graph and a conversion formula that converts raw ADC values into motor speed.

5. Control Strategy

We decided that we will use an ADC to set a speed value for both motors by rotating, then use a Python script to map the WASD keys for up down, left and right to the motor target direction for more consistent movement of the target.

6. Speed Range Selection

Appropriate speed or distance settings (What is your min? What is your max?)

The min speed value is 0. A value of 0 corresponds to a stopped motor, and motor motion begins on any value above 0, depending on the axis and mechanical load. The max speed value is 7000 for the horizontal motor and 70000 for the vertical motor (the register value). The reason why there is a difference is because the vertical motor uses a lead screw which takes more rotations for vertical translation. A lower maximum was used for the horizontal axis because its shorter travel and lower displacement requirement made high speeds less useful for controlled motion in this motion type.

The max speed you can set to the motor in hex is 0xFFFFF, which equates to 1048575 in decimal (which can be converted to steps per second using the conversion factor). However, we only use up to 70000 as this is a reasonable max speed for operation purposes (being able to move the motor with enough precision such that it isn't too fast to control). These maximum values were selected experimentally for each axis by gradually increasing the speed during testing until the motion became less stable, less controlled, or less suitable for precise operation, then choosing the highest values that still gave stable and usable motion.

Motor vibration/jittering problem:

We discovered that while testing the motors speed changing using the potentiometer, the motors would start to jitter after turning the output on or off. Initially we speculated that the motor may be faulty and after a certain speed the motor starts to malfunction.

After doing some research on stepper motors and seeking help from the TAs we

understood that as we turned the output on or off from the power supply we did not reset the motor shield, so the old driver of the motor was not updated as the power was cut and then turned on. Now every time the power is cut, and turned on, the motor shield is reset before starting the motors.

7. Direction Control and Safety

How can you safely handle direction changes?

To safely handle direction changes, the system performs the following steps:

1. Stop the motor
2. Apply a no interrupt delay (for loop)
3. Restart the motor in the desired direction

Initially, a soft stop was used. However, during testing, it was observed that:

- the motor overshoot the desired stopping position when trying to accurately position the platform
- the limit switch logic sometimes failed due to the slow stopping response

This behavior occurs because the soft stop gradually reduces speed, which delays the stopping action and allows the system to move beyond the intended position.

To address this, the system was modified to use a hard stop before changing direction. This ensures that the motor stops immediately before reversing direction.

Since the system operates at relatively low speeds, the hard stop is not too aggressive and does not degrade motor performance.

This approach ensures reliable and safe motor operation, particularly when interacting with limit switches, and demonstrates that control strategies must be adapted based on system dynamics and safety requirements.

8. Operating Modes

Edge cases of operation and how you will test them

Two modes of operation were implemented:

- Normal Mode: In normal mode, the motors continuously move at speeds determined by the ADC conversion from the potentiometer input. Direction changes

occur only when limit switches are triggered, ensuring safe operation within the physical boundaries of the system.

- **Manual Control Mode:** Manual control mode is activated using keyboard input (wasd and J/K keys) sent through the serial port, or a Python script GUI that sends commands such as (XS (x-stop), XP (x-positive), XN (x-negative), and same for y). The script sends serial UART messages over the COM port to control motor direction. It also allows stopping motion in both the horizontal and vertical axes (when activated, it sends the stop command for both directions every 0.5 second when it should be stopped to ensure motors are stopped if they started moving for any reason, and ensure the system is put in manual mode). Manual control mode could be exited (going back to normal mode) if the command NRMLCNTRL was received through UART.

The edge cases for motor control are the following:

- 1) In manual mode, if the user commands the motor to move in one direction (while holding the keyboard key) and it hits a limit switch, then the logic would disable and ignore all manual inputs (keyboard keys) and rotate the motor away from the limit switch until the limit switch is not pressed anymore. Once the limit switch is not pressed, manual inputs is enabled once again. This prevents the user from trying to go past the limit switch. (when switch is triggered, both directions are stopped, and any keyboard button that was pressed before the limit switch is ignored and has to be repressed to take effect).
- 2) If both limit switches in a given axis are pressed, then the state machine goes into fault mode, which is indicated by an LED (connected on pin PB8 D15, as an output pin). The state does not go out of fault mode until the system is reset (reset button on the microcontroller is pressed).
- 3) If the switch is pressed while it is already going in the same direction that the switch sets it to, then in normal mode, it ensures it is going by sending the run command only (no stopping).

9. Synchronization issue encountered during testing and implemented fix:

At the end of the lab, it was observed that the interaction between limit switch interrupts and ADC-based speed control caused some inconsistent behavior in:

- motor direction switching
- stopping at limit positions

This is likely due to timing conflicts between continuous ADC polling and interrupt execution, and may require further synchronization improvements.

With debugging, it was determined that due to the race condition of the ADC function with the interrupts, where the interrupt could be called after the motor direction was obtained by the ADC function, therefore, as soon as the interrupt is done, switching the direction, the continuation of the polling code would return the motor to the previous direction. To solve this issue, a global variable for motor speed, and another for motor stopped or not were created, and they were modified everywhere there are changes to motor direction or stopping. These variables were used in the ADC speed setting function, instead of reading the direction register of the motor driver, and a debounce time was added, then the direction in the variable was compared to the direction on the register, which can catch any issues caused by the interrupt call in the middle of the direction change, and set it to the interrupt's direction. This ensures that limit switch direction is always the direction the motor will go.

Another issue was that sending the UART command that returns the motor to the normal mode (NRMLCNTRL) would always reset the motors, and make them go to the initial random directions of the initialization function. To solve this problem, a check was added to ensure that the code does nothing if it is already in normal control. Also, the direction of the motor initialization function (if not switch was pressed) was made to not be random, but was set to the previous direction that was on the motor direction global variable.

Note: the global variables mentioned were created one for each motor (array of motor_0 and motor_1 items).

10. Python Manual Control script summary

The Python GUI script, which can be found in Keyboard_CNTRL.py, performs a simple task: it sends commands to the STM32 and shows the user which commands are being sent. It sends the commands below:

- XS/YS: x/y-stop.
- XP/YP: x/y-positive direction.
- XN/YN: x/y-negative direction.
- NRMLCNTRL: go back to normal control mode.

The GUI allows the user to select the COM port and baud rate, start or stop manual control, and view visual feedback such as the pressed keys, current motion direction, last command sent, and any serial communication errors. When the GUI starts and remains active, it sends the XS and YS commands every 0.5 s so the STM32 stays in manual mode until the NRMLCNTRL command is sent. When a keyboard button is pressed, the corresponding stop command for that axis is no longer resent, and the movement command is transmitted twice with a short delay to improve reliability. If contradictory keys are pressed, the GUI treats the input as a stop condition and continues sending the stop

command. Overall, the GUI provides the user with basic operating instructions, displays the expected direction of motion, shows the commands being transmitted and the keys currently pressed, and manages switching between manual and normal control while maintaining the serial connection to the selected port.

11. Summary

The system successfully integrates ADC-based speed control, interrupt-driven limit switch safety, and manual user control. This enables flexible operation while maintaining safe and stable motor behavior across both axes.

12. Practicing the target consistency before the demo:

1 st	45.30
2 nd	45.90s
3 rd	47.80

13. References and Resources:

Resources used for this lab included the schematics and function documentation provided on Learn, lecture material, TA assistance during debugging and testing, team members' prior embedded programming knowledge, and the STM32F MCU datasheet and reference documentation linked through Learn for ADC settings, channels, and timing details. Any additional resource use that directly affected implementation decisions was documented in the relevant section.

Appendix

Lab old version:

Initial approach is to detect the falling edge of the limit switch, trigger an interrupt and stop the motor.

Choosing the falling edge since the switch is normally closed, the switch will be connected to the input 3.3V from the Arduino connected to the motor, when the switch opens a falling edge is triggered and the interrupt will happen then.

(Yaseen and ## did the wiring)

We set the power supply 12V and limit current will be 1A.

(##, ## and ## looked at schematics to find empty pins)

Determine the pin that free to use. First step: check on the code if that pin is already used for other purposes. After that, we check on the motor shield to see if that pin have been used or not. If not, that pin is good to use. Observation: The Pin with B is more available than Pin with A and C. Eventually, we find: PA8, PA9, PA10, PB8, PB9, PB10....are available.

First step is to know that we will only use the pins on the motor shield, then cross referencing from the motor shield schematic, we identified that PB6, PA9, PA10, PA11 are free on the motor shield and can be used. These are the pins we will be using.

In the code we created a new function that handles lines 8 and 9 and 6.

(everyone helped write the code)

@brief This function handles EXTI Line[9:5] interrupts.

*/

void EXTI9_5_IRQHandler(void)

{

if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_9) != RESET)

{

```
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_9);
}
if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_8) != RESET)
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_8);
}
if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_6) != RESET)
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_6);
}
}
```

(Yaseen and ## tested the unmounted motors with the limit switches)

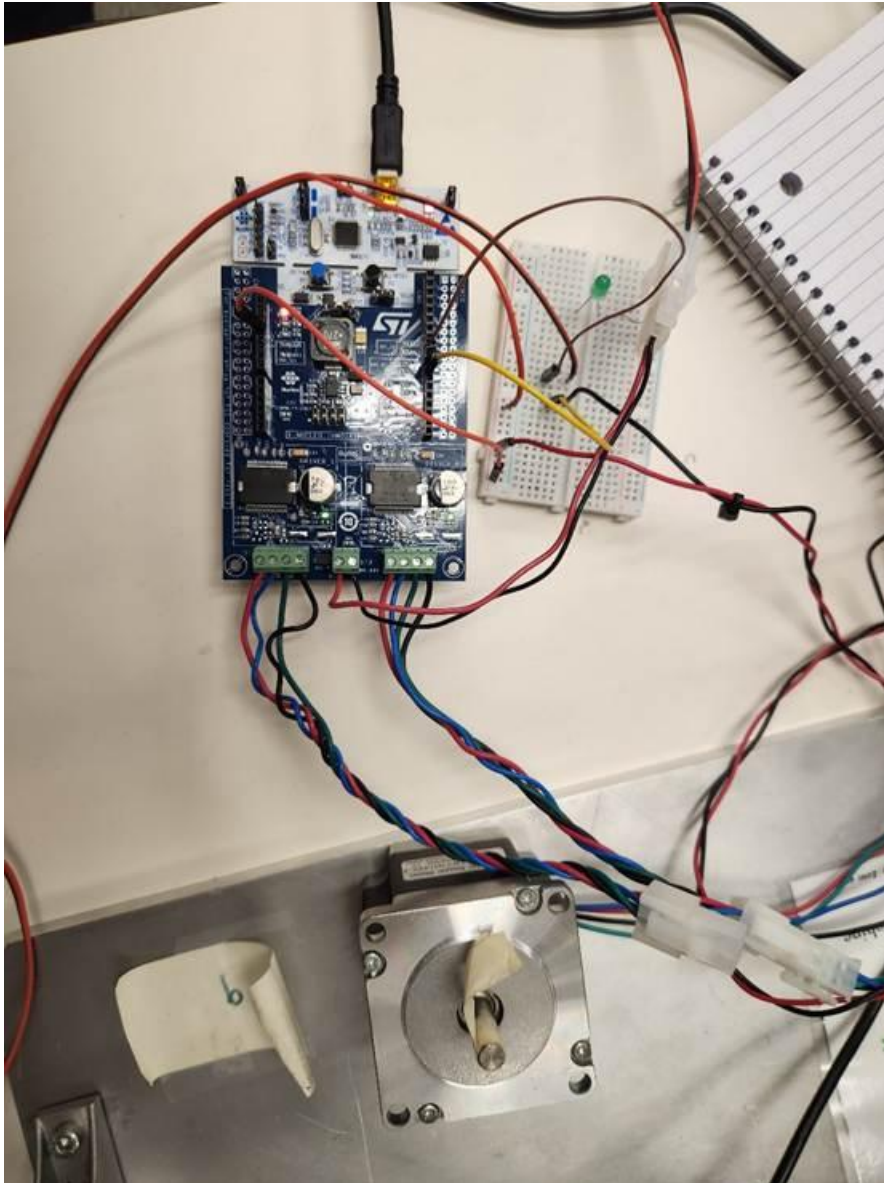
Then set priority of all pins to sub priority 0, which means highest priority.

The first step, we connect the unmounted motor to test if we can rotate it and when hit limit switch if it turn on or not

(## and ## brainstormed testcases and how to implement them)

Edge cases to consider:

- First case is if 2 limit switches are pressed at the same time (already accounted for)
- If the program flashes and starts off with a limit switch being pressed



Adding the interrupt logic.

Declaring pin A8, A9, A10, A11 as falling edge triggered interrupts in the interface.c file

```
{  
#ifdef NUCLEO_USE_USER_BUTTON  
    /* Configures Button GPIO and EXTI Line */  
    BSP_PB_Init(BUTTON_KEY, BUTTON_MODE_EXTI);  
#endif
```

```
#ifdef NUCLEO_USE_USER_LED
```

```
/* Configures LED GPIO */
```

```
BSP_LED_Init(LED2);
```

```
#endif
```

```
__HAL_RCC_GPIOA_CLK_ENABLE();
```

```
__HAL_RCC_GPIOB_CLK_ENABLE();
```

```
/* Declare a struct, which is then to be specified to configure an GPIO Pin */
```

```
GPIO_InitTypeDef GPIO_InitStructure = {0};
```

```
GPIO_InitStructure.Pin = GPIO_PIN_8; // Change X to your pin
```

```
GPIO_InitStructure.Mode = GPIO_MODE_IT_FALLING;
```

```
GPIO_InitStructure.Pull = GPIO_PULLDOWN; // Use pulldown when the switch is connected to 3.3V
```

```
GPIO_InitStructure.Speed = GPIO_SPEED_FREQ_LOW;
```

```
HAL_GPIO_Init(GPIOA, &GPIO_InitStructure); // Sends the configuration to specified Port and Pin settings
```

```
HAL_NVIC_SetPriority(EXTI9_5_IRQn, 2, 0);
```

```
HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);
```

```
GPIO_InitTypeDef GPIO_InitStructure_1 = {0};
```

```
GPIO_InitStructure_1.Pin = GPIO_PIN_9; // Change X to your pin
```

```
GPIO_InitStructure_1.Mode = GPIO_MODE_IT_FALLING;
```

```
GPIO_InitStructure_1.Pull = GPIO_PULLDOWN; // Use pulldown when the switch is connected to 3.3V
```

```
GPIO_InitStructure_1.Speed = GPIO_SPEED_FREQ_LOW;
```

```
HAL_GPIO_Init(GPIOA, &GPIO_InitStructure_1); // Sends the configuration to specified Port and Pin settings
```

```
HAL_NVIC_SetPriority(EXTI9_5_IRQn, 2, 0);
```

```
HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);
```

```
GPIO_InitTypeDef GPIO_InitStruct_2 = {0};
```

```
GPIO_InitStruct_2.Pin = GPIO_PIN_6; //Change X to your pin
```

```
GPIO_InitStruct_2.Mode = GPIO_MODE_IT_FALLING; //Push-Pull output, meaning the pin  
supplies 3.3V and 0V
```

```
GPIO_InitStruct_2.Pull = GPIO_PULLDOWN; //No internal pullup/down for output
```

```
GPIO_InitStruct_2.Speed = GPIO_SPEED_FREQ_LOW;
```

```
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct_2);
```

```
HAL_NVIC_SetPriority(EXTI15_10_IRQn, 2, 0);
```

```
HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
```

```
GPIO_InitTypeDef GPIO_InitStruct_3 = {0};
```

```
GPIO_InitStruct_3.Pin = GPIO_PIN_10; //Change X to your pin
```

```
GPIO_InitStruct_3.Mode = GPIO_MODE_IT_FALLING; //Push-Pull output, meaning the pin  
supplies 3.3V and 0V
```

```
GPIO_InitStruct_3.Pull = GPIO_PULLDOWN; //No internal pullup/down for output
```

```
GPIO_InitStruct_3.Speed = GPIO_SPEED_FREQ_LOW;
```

```
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct_3);
```

```
HAL_NVIC_SetPriority(EXTI15_10_IRQn, 2, 0);
```

```
HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
```

```
}
```

We can connect the second motor and using PB6 and PB10 for interruption

The next step is checking the edge cases in main, on startup. This is done to avoid any edge cases being missed and causing unexpected behavior on running. Edge cases being accounted for are: Yaseen and ##

- Both limit switches on one of the Axes are pressed.
 - That indicates possible faulty hardware, connections, or software issue that caused this detection.
 - The solution was to fully stop both motors and set a flag to indicate the fault and avoid any motor being driven, and print an error serially, allowing the user to debug and fix the fault, and avoid any equipment damage.
- One of the limit switches is pressed on startup.
 - That indicates the motor is at one of the two sides, therefore, it starts moving in the opposite direction (assuming the position of switches and required direction after triggering each is known).

Note: if any fail cases is triggered (both switches triggered on the same axis)

Code:

```
{  
/*read all limit switch values*/  
  
GPIO_PinState SW1_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_10);  
GPIO_PinState SW2_value = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_6);  
GPIO_PinState SW3_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9);  
GPIO_PinState SW4_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8);  
  
bool motor_0_set = false;  
bool motor_1_set = false;  
  
//ensure fault is reset and not detected at the start of the program  
fault_detected = false;  
  
// horizontal axis
```



```
if (SW3_value == GPIO_PIN_RESET && SW4_value == GPIO_PIN_RESET) //motor 0 both switches triggered
```

Will modify it more

```
{
    L6470_Debounce_Delay(debounce_delay_ms);

    SW3_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9);
    SW4_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8);

    if (SW3_value == GPIO_PIN_RESET && SW4_value == GPIO_PIN_RESET) //motor 0 both switches still triggered after debounce delay
    {
        L6470_HardStop(0);
        L6470_HardStop(1);
        fault_detected = true;

        USART_Transmit(&huart2, "Fault detected: both motor 0 switches triggered fix issue and reset\r\n");

        motor_0_set = true;
        motor_1_set = true;
    }
}

if (SW3_value == GPIO_PIN_RESET && motor_0_set == false && fault_detected == false) //motor 0 right switch triggered PA9
{
    L6470_Debounce_Delay(debounce_delay_ms);

    SW3_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9);

    if (SW3_value == GPIO_PIN_RESET) //motor 0 right switch still triggered after debounce delay
```

```

{
    L6470_Run(0, L6470_DIR_FWD_ID, 3000);

    motor_0_set = true;
}
}

if (SW4_value == GPIO_PIN_RESET && motor_0_set == false && fault_detected == false)
//motor 0 left switch triggered PA8

{
    L6470_Debounce_Delay(debounce_delay_ms);

    SW4_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8);

    if (SW4_value == GPIO_PIN_RESET) //motor 0 left switch still triggered after debounce
delay
    {
        L6470_Run(0, L6470_DIR_REV_ID, 3000);

        motor_0_set = true;
    }
}

if (motor_0_set == false && fault_detected == false) //motor 0 no switch triggered

{
    L6470_Run(0, L6470_DIR_FWD_ID, 3000);

    motor_0_set = true;
}

// vertical axis

if (SW1_value == GPIO_PIN_RESET && SW2_value == GPIO_PIN_RESET && fault_detected
== false) //motor 1 both switches triggered

{

```

```

L6470_Debounce_Delay(debounce_delay_ms);

SW1_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_10);

SW2_value = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_6);

if (SW1_value == GPIO_PIN_RESET && SW2_value == GPIO_PIN_RESET) //motor 1 both
switches still triggered after debounce delay

{

    L6470_HardStop(1);

    L6470_HardStop(0);

    USART_Transmit(&huart2, "Fault detected: both motor 1 switches triggered fix issue and
reset\r\n");

    fault_detected = true;

    motor_0_set = true;

    motor_1_set = true;

}

}

if (SW1_value == GPIO_PIN_RESET && motor_1_set == false && fault_detected == false)
//motor 1 top switch triggered PA10

{

    L6470_Debounce_Delay(debounce_delay_ms);

    SW1_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_10);

    if (SW1_value == GPIO_PIN_RESET) //motor 1 top switch still triggered after debounce
delay

    {

        L6470_Run(1, L6470_DIR_FWD_ID, 30000);

        motor_1_set = true;

    }

}

```

```

if (SW2_value == GPIO_PIN_RESET && motor_1_set == false && fault_detected == false)
//motor 1 bottom switch triggered PA6

{
    L6470_Debounce_Delay(debounce_delay_ms);

    SW2_value = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_6);

    if (SW2_value == GPIO_PIN_RESET) //motor 1 bottom switch still triggered after debounce
delay
    {
        L6470_Run(1, L6470_DIR_REV_ID, 30000);

        motor_1_set = true;
    }
}

if (motor_1_set == false && fault_detected == false) //motor 1 no switch triggered
{
    L6470_Run(1, L6470_DIR_FWD_ID, 30000);

    motor_1_set = true;
}
}

```

Next, the interrupted handler was setup for each pin to set the direction of the motor opposite to that switch (this assumes the position vs direction of motor is known and correct for each motor). This was done using the code below: ## and Yaseen

```

{
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    switch (GPIO_Pin)
    {

```

```
case GPIO_PIN_9: // horizontal axis right switch

    L6470_Debounce_Delay(debounce_delay_ms);

    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_9) == GPIO_PIN_RESET && fault_detected ==
false)

    {

        L6470_Run(0, L6470_DIR_FWD_ID, 3000);

    }

    break;

case GPIO_PIN_8: // horizontal axis left switch

    L6470_Debounce_Delay(debounce_delay_ms);

    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_8) == GPIO_PIN_RESET && fault_detected ==
false)

    {

        L6470_Run(0, L6470_DIR_REV_ID, 3000);

    }

    break;

case GPIO_PIN_10: // vertical axis top switch

    L6470_Debounce_Delay(debounce_delay_ms);

    if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_10) == GPIO_PIN_RESET && fault_detected ==
false)

    {

        L6470_Run(1, L6470_DIR_FWD_ID, 30000);

    }

    break;

case GPIO_PIN_6: // vertical axis bottom switch

    L6470_Debounce_Delay(debounce_delay_ms);
```

```

    if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_6) == GPIO_PIN_RESET && fault_detected ==
false)
    {
        L6470_Run(1, L6470_DIR_REV_ID, 30000);
    }
    break;
case GPIO_PIN_13:
    BSP_EmergencyStop();
    break;
case L6470_nBUSY_SYNC_GPIO_PIN:
    BSP_L6470_BusySynchEventManager();
    break;
case L6470_nFLAG_GPIO_PIN:
    BSP_L6470_FlagEventManager();
    break;
}
}
}

```

Where the handler call was initialized using this code:

```

{
void EXTI15_10_IRQHandler(void)
{
    if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_10) != RESET)
    {
        HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_10);
    }
}

```

```
if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_13) != RESET)
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_13);
}
}
```

```
/**
```

```
* @brief This function handles EXTI Line[9:5] interrupts.
```

```
*/
```

```
void EXTI9_5_IRQHandler(void)
```

```
{
    if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_9) != RESET)
    {
        HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_9);
    }
    if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_8) != RESET)
    {
        HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_8);
    }
    if (__HAL_GPIO_EXTI_GET_IT(GPIO_PIN_6) != RESET)
    {
        HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_6);
    }
}
}
```

This code uses the L6470_Run function to control the direction and speed of the motors. This function has an internal check for the fault detected Boolean that ensures the motor does not run if it is in this mode, to avoid equipment damage.

Lab 4 session:

Part 1:

From lab 3 code, add a debounce to the limit switches (15ms delay then check switch again), to ensure the switch is actually triggered, and cleanup noise in the clicks. The process is:

- Delay 15ms
- Read button values again
- Check if still triggered, and apply conditions if that is the case.

Note: this is applied to edge cases, as well as interrupts.

Example interrupt debounced code:

```
{  
case GPIO_PIN_10:  
    HAL_Delay(SW_debounce_delay_ms); //delay to ensure the switch value is stable  
    GPIO_PinState switch_value = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_10);  
    if (switch_value == GPIO_PIN_RESET)  
    {  
        L6470_Run(1, L6470_DIR_REV_ID, 3000);  
    }  
    break;  
}
```

After testing and implementing this code, we found that the HAL_Delay function caused the code to break as it itself is also interrupt driven and was causing interference with our switch interrupts. For the demo we switched to a pure stopping the motor case instead of reversing the direction.

Yaseen and ##

We implemented cases where we check the value of any switch, if it is pressed, we stop the corresponding motor, the 2 switches on x axis correspond to one motor and the 2 on the y axis correspond to the other motor. We also are not implementing any debouncing for this approach as in the debouncing we would check if any given switch were pressed, then wait, and check again just to stop. Without the debouncing we just receive a signal and stop the motor, even with debouncing we would just check and stop again.

and

We then worked on implementing the logic to reverse the motor direction after a switch click. The main logic follows that we know the location of each switch and the direction desired when it is pressed. Switches 1 to 4 are hardcoded locations, we setup the interrupt cases for each switch to move the corresponding motor in the opposite direction from that switch rather than toggling. This mostly eliminates the need for debouncing as at any slight switch trigger the motor is set to move in the away direction from the button (does not matter if it was one press or multiple), now when running the code while it is at any corner this case is handled as well, since both motors would just go in the opposite corner at any switch trigger. (the code and cases matches the final implementation explained in lab 3)

Then there was a possibility of any noise on the lines being seen as a momentary button press and switching the direction of the motor unintentionally, which is a very unlikely case, but it is better to be handled now. To fix this, a debounce was added, with a custom L6470_Debounce_Delay function added to the stepper motors' functions, which uses a for loop to loop for 20000 times for each millisecond, approximating the delay and not interfering with the interrupts, to avoid all delay interrupt issues:

```
void L6470_Debounce_Delay(uint32_t delay_ms){  
    uint32_t t = delay_ms*20000;  
    for (int i = 0; i < t; i++) {}  
};
```

This was then implemented in the debounce for the buttons as follows:

```
case GPIO_PIN_10: // vertical axis top switch  
    L6470_Debounce_Delay(debounce_delay_ms);
```

```
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_10) == GPIO_PIN_RESET && fault_detected ==  
false)  
{  
    L6470_Run(1, L6470_DIR_FWD_ID, 30000);  
}  
break;
```

This lead to buttons being debounced, and will only be triggered when actually pressed, not with any outside noise.

To add: add a check when it is fired to check if the other switch in that axis is also pressed, as in such a case, there is an issue with the system and it needs to completely stop all motors for safety of equipment.

Part 2:

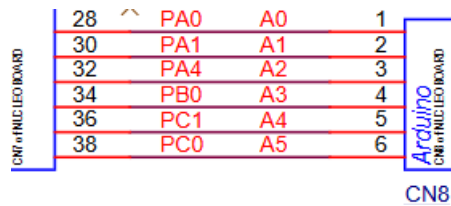
ADC Work

We figured out how to connect the Potentiometer to board as the picture showing below.

##



We found the ADC pin



We use PB0

Steps:

1. Activate Internal ADC

- a. What is needed to turn it on?:
 - i. Initially, uncomment the config code in MX_ADC1_Init(void) function (explained below)
 - ii. To start ADC reading, use HAL_ADC_Start function to enable the ADC and start conversion of regular channels

The code we modified for the ADC configurations, explanation provided later:

and

```
void MX_ADC1_Init(void)
{
    ADC_ChannelConfTypeDef sConfig;

    /* GPIO Ports Clock Enable */
    __GPIOB_CLK_ENABLE();

    /**Configure the global features of the ADC (Clock, Resolution, Data Alignment and
    number of conversion)
    */
    hadc1.Instance = ADC1;
    hadc1.Init.ClockPrescaler = ADC_CLOCKPRESCALER_PCLK_DIV4;
    hadc1.Init.Resolution = ADC_RESOLUTION_8B;
    hadc1.Init.ScanConvMode = DISABLE;
    hadc1.Init.ContinuousConvMode = ENABLE;
    hadc1.Init.DiscontinuousConvMode = DISABLE;
    hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
    hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
    hadc1.Init.NbrOfConversion = 1;
    hadc1.Init.DMAContinuousRequests = DISABLE;
    hadc1.Init.EOCSelection = ADC_EOC_SEQ_CONV;
    HAL_ADC_Init(&hadc1);
}
```

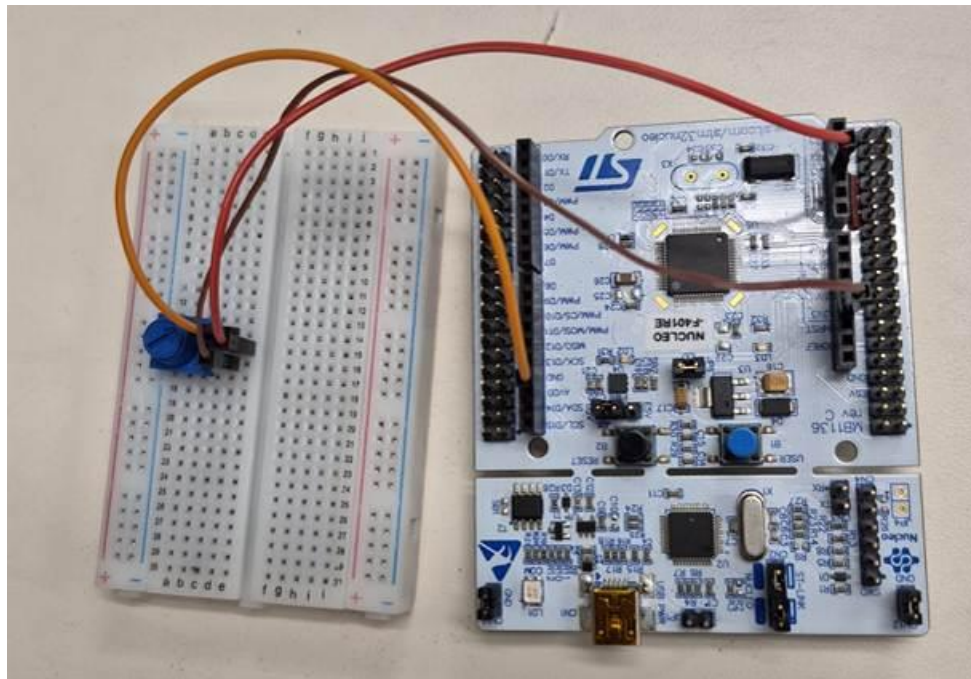
```

/**Configure for the selected ADC regular channel its corresponding rank in the
sequencer and its sample time.
*/
sConfig.Channel = ADC_CHANNEL_8;
sConfig.Rank = 1;
sConfig.SamplingTime = ADC_SAMPLETIME_15CYCLES;
HAL_ADC_ConfigChannel(&hadc1, &sConfig);
}

```

2. Connect your potentiometer

- a. After looking at the schematic, PB0 was found to be an analog pin that was not already used. Thus, it was selected as our ADC input pin.



b.

3. Configure your internal ADC

What resolution did you choose? Why?

The Nucleo board supports a 6-, 8-, 10-, and 12-bit resolution for the ADC. Each increment of bit provides a 2^N number of digital steps that can be represented with a reference input analog voltage. In this case we are using a very small potentiometer with a very small knob that must be turned by hand to increase or lower the speed of the motor which is sliding or moving across a very small distance. The first consideration that narrows down the choice of number of bits is how many unique positions the potentiometer knob can we step through

using our hands, in other words how small of a change in the resistance can we create with our hands over the 10k ohm resistance of the potentiometer. 12-bit gives 4096 unique positions, too much for a human hand to replicate on such a small knob, 10-bit gives 1024, still in the thousands, too high for human hands. 8-bit offers 256 unique positions and 6-bit offers 64, both of which are plausible and achievable in terms of human hand increments. The next factor to consider is the small length that the motor moves over, a 64 bit resolution would provide a choppy/steep change in speed if the potentiometer position is changed quickly, possibly resulting in erratic movement of the motor from slow to very fast and vice versa, leading to malfunction or damage. While 8-bit offers a considerably smoother transition in between motor speeds. 8-bit provides the best mix of needed resolution in the context of the human controller and the required smoothness of speed change on the motors.

What conversion mode? Why?

The continuous conversion mode was set to enable sampling to happen continuously instead of only sampling once or manually at a specific time.

What is your sampling time? Why?

We configured the ADC1 sampling time to 15 cycles. At 21 MHz, one cycle is 47.6ns. If a sampling time of 3 cycles is used, then the total sampling time is 142.8ns. The following calculations were done to find the required time for an 8-bit accuracy, while assuming the voltage must settle within 0.5 LSB:

$$V_c(t) = V_{in} \left(1 - e^{-\frac{t}{\tau}}\right) V_{error} = V_{in} - V_c(t) = V_{in} \left(e^{-\frac{t}{\tau}}\right)$$

$$\frac{V_{error}}{V_{in}} = e^{-\frac{t}{\tau}} = \frac{0.5}{2^n}$$

$$t = 6.24 \tau$$

This means that the RC circuit needs about 6.2 time constants to settle within 0.5 LSB for 8-bit accuracy.

Now solving for the time constant, the max is used, based on the full resistance value from the potentiometer (10kohm) being in series with the internal resistance of (6kohm), totalling 16kohm:

$$\tau = RC = 16000ohm * (4 * 10^{-12}Farad) = 64ns$$

For 8-bit accuracy, we require approximately 6.2 time constants to allow the internal capacitor to charge within 0.5 LSB of the input voltage.

$$64ns * 6.2 = 384ns$$

Now, since the sampling time of 3 cycles (142.8ns) is less than the calculated required time for the voltage to settle, this means that the sampling time must occur over a greater number of cycles. The next sampling time cycle constant available in the HAL is ADC_SAMPLETIME_15CYCLES. With 15 cycles instead, the sampling time becomes 714ns, which is greater than the required time of 384ns. By choosing 15 cycles (714ns), we ensured the internal capacitor has more than enough time to fully charge, which prevents 'ghosting' or lagged readings when we adjust the motor speed in real-time

What happens when you change your clock settings?

The ADC clock prescaler was set to divide the APB2 clock by four. According to the datasheet of the STM32F401xE, the max frequency of the ADC is 32 MHz. The main clock frequency is 84 MHz on APB2. Using a prescaler of 2 would mean $84/2 = 41$ MHz which is too high and exceeds the limit. Thus, a prescaler of 4 is used which will be $84/4 = 21$ MHz which fits the requirement. This produces an ADC clock that remains within the recommended operating range while still allowing fast conversions. Because the potentiometer input changes slowly, and only an 8-bit resolution is required for motor control, extremely high ADC speeds are unnecessary. Using a prescaler of four provides a balance between conversion speed and measurement stability.

How can you ensure your ADC is enabled and working?

Now inside the main loop, we add the condition to check if the ADC conversion is complete, then read its value (0-255). The read return value is printed to ensure the range of values is properly read and convert to motor speed value later. The code below was used in main to read and print the ADC value:

```
{  
HAL_ADC_Start(&hadc1);  
  
volatile uint16_t ADC_raw = 0;  
  
volatile uint16_t ADC_old = -1;  
  
HAL_StatusTypeDef testing;  
  
  
/* Infinite loop */
```

```

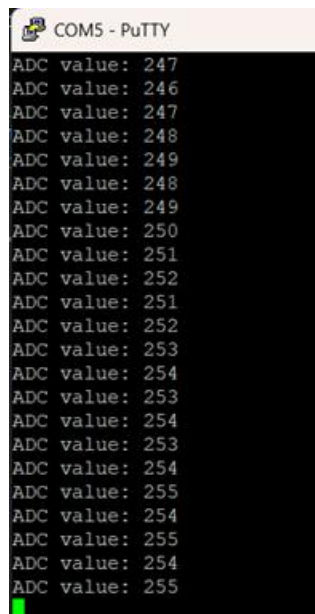
while (1)
{
    /* Check if any Application Command for L6470 has been entered by USART */
    // USART_CheckAppCmd();

    // tight polling for ADC conversion, if the conversion is complete then read the value and
    store in ADC_raw variable

    testing = HAL_ADC_PollForConversion(&hadc1, 10);
    if (testing == HAL_OK)
    {
        // read the ADC value and store in ADC_raw variable
        ADC_raw = (uint16_t)HAL_ADC_GetValue(&hadc1);
Yaseen
        if (ADC_raw!=ADC_old) //only transmit the new ADC value if it is different from the previous
        value to avoid unnecessary UART transmissions
        {
            ADC_old = ADC_raw;
            USART_Transmit(&huart2, "ADC value: ");
            char buffer[10];
            sprintf(buffer, "%d", ADC_raw);
            USART_Transmit(&huart2, buffer);
            USART_Transmit(&huart2, "\r\n");
        }
    }
}
}

```

After verifying the ADC value is read properly, and printed as shown below



```
COM5 - PuTTY
ADC value: 247
ADC value: 246
ADC value: 247
ADC value: 248
ADC value: 249
ADC value: 248
ADC value: 249
ADC value: 250
ADC value: 251
ADC value: 252
ADC value: 251
ADC value: 252
ADC value: 253
ADC value: 254
ADC value: 253
ADC value: 254
ADC value: 253
ADC value: 254
ADC value: 255
ADC value: 254
ADC value: 255
ADC value: 254
ADC value: 255
```

The prints were removed, and the ADC value was converted to a speed value range for the vertical motor (10000 to 50000), and the motor speed was set as a global variable, and the motor speed would immediately change to the new speed as the ADC value changes.

As the potentiometer resistance is changed that results in the equivalent change in the ADC output which has been mapped linearly to an output between 0 and 255.

Ideally your ADC is linear. How accurate is it?

For calibration of ADC, we have V_Potentiometer vs ADC out graph.

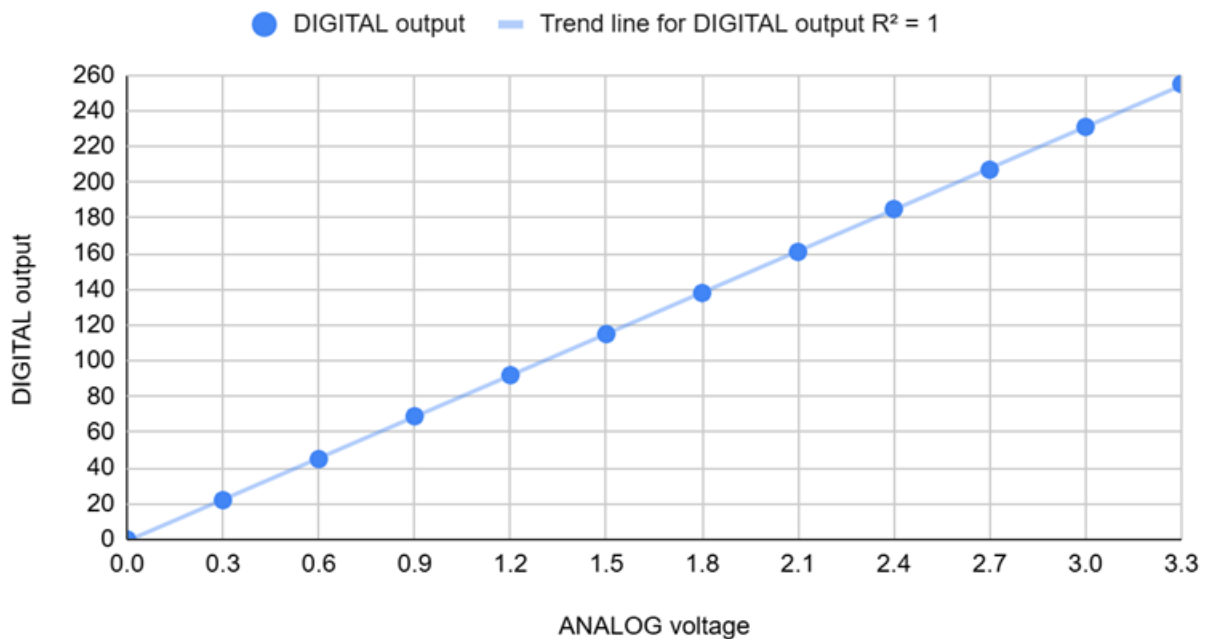
Testing the ADC linearity to check if it's working.

Taking 11 data points from Voltage of analog.

Measuring the ADC output value at each step.

This shows that the characterization curve of our ADC is linear to what and R^2 value of 1. It is very accurate. The ADC was configured with 8-bit resolution and a reference voltage of 3.3 V. This results in 256 discrete quantization levels. The voltage resolution of the ADC is therefore $3.3V/256 \approx 12.9mV$ per step.

DIGITAL output vs. ANALOG voltage

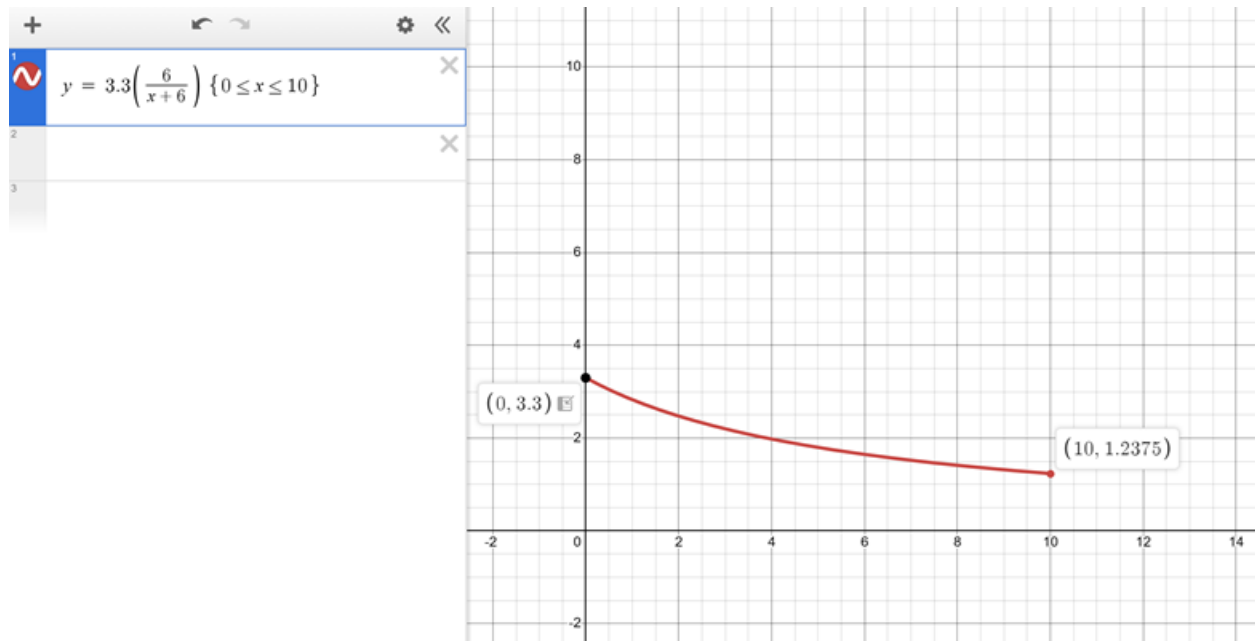


What is the error in your ADC?

The theoretical quantization error of an ADC is ± 0.5 LSB. For this configuration, this corresponds to approximately ± 6.45 mV. This represents the inherent measurement uncertainty of the ADC due to quantization.

The experimental results showed a linear relationship between the potentiometer voltage and the digital ADC output, indicating that the ADC behaved as expected within the limits of quantization error. Since the measured response was linear, no significant integral nonlinearity (INL) was observed within the experimental resolution.

For potentiometer characterization, it is a voltage divider circuit, graph v pot_resistance



Define and carry out a calibration procedure? Does it behave linearly? How can you map the ADC readings to motor speed?

To calibrate the ADC we will make use of the voltage input vs digital output graph and interpolate if we need to go to a value in between the sampled data points. Then we can use the potentiometer resistance(x) vs potentiometer voltage (y) in the above graph. This provides a linear relationship between a change in potentiometer resistance and a change in the ADC output which is mapped to speed (in below code), we map the raw ADC value read to between 10000 (min motor speed) and 50000 (max motor speed), by reading raw ADC output multiplying by the difference of 50000-10000 divided by 255 and added with 10000, this gives a speed of 10000 at ADC out = 0 and 50000 at ADC out = 255 (highest and lowest speeds possible) and all the steps of speed in between mapped linearly to the analog input changed through the potentiometer, this allows us to rotate the potentiometer knob and increase or decrease the motor speed.

After ADC:

A global variable was added for the motor speed, and the ADC polling was slightly modified to modify the motor speed, while it is going in the same direction. The code reads the ADC value, then calculates the speed using $\text{ADC_raw} * (10000 - 50000) / 255 + 10000$. It then does

the limit switch checks from lab3, and if a motor will be moved, it turns it on with that speed (the interrupts also move the motor with that speed when changing direction). Then the while loop in main runs continuously, and in each cycle it reads this ADC value, compares it to the previous value, and if it changed, it sets it as the new motor speed variable, and checks if fault was not detected (motors are halted to avoid damage if fault detected) and if motor is in motion (BUSY_ID is 0), then it moves the motor with that same speed, in the direction it was moving (uses the function L6470_CheckStatusRegisterFlag(1, DIR_ID) to get motor direction).

Code:

```
{
while (1)
{
    /* Check if any Application Command for L6470 has been entered by USART */
    USART_CheckAppCmd();

    // tight polling for ADC conversion, if the conversion is complete then read the value and
    store in ADC_raw variable

    testing = HAL_ADC_PollForConversion(&hadc1, 10);
    if (testing == HAL_OK)
    {
        // read the ADC value and store in ADC_raw variable
        ADC_raw = (uint16_t)HAL_ADC_GetValue(&hadc1);

        if (ADC_raw!=ADC_old) //only transmit the new ADC value if it is different from the
        previous value to avoid unnecessary UART transmissions

        {
            ADC_old = ADC_raw;

            motor_1_speed = 10000 + ADC_raw*(50000-10000)/255; // map the ADC value which is
            between 0 and 255 to a motor speed value between 10000 and 50000
```

```
    if (fault_detected == false && L6470_CheckStatusRegisterFlag(1, BUSY_ID) == 0) //only  
    update the motor speed if a fault condition has not been detected and motor is already in  
    motion (BUSY_ID flag is 0 when motor is in motion)
```

```
{
```

```
    L6470_Run(1, L6470_CheckStatusRegisterFlag(1, DIR_ID), motor_1_speed); //update  
    motor 1 speed with the new value read from the ADC
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```